

Instructions du montage du Raspberry Pi 4

Instructions étape par étape permettant la construction de la configuration du Raspberry Pi 4 :

La carte SD étant vierge, on commence par l'installation de l'image de Raspberry Pi 4 :

Dans un premier temps, il faut installer l'image de Raspberry Pi 4 sur une carte micro SD. Connectez la carte à un pc possédant le logiciel Raspberry Pi Imager. Ensuite, on sélectionne l'image de Raspberry Pi 4 et on suit les instructions. Une fois l'image installée, on peut placer la carte SD dans le Raspberry Pi, puis le brancher à une alimentation, un écran, un clavier, une souris et à un câble ethernet.

Une fois lancé, on continue le paramétrage, on entre ces informations :

Nom d'hôte : stigtest (nom que nous avons utilisé provisoirement, il peut être modifié à tout moment, c'est personnalisable)

Ville : Paris / Fr (Entrez ville locale)

Compte utilisateur : simon1 / mdp : iotcompose1 (Personnalisable)

SSID : SSID_test_simon (Modifiable à tout moment)

MDP : MDP_test_simon (Modifiable à tout moment)

Ip Raspberry : 192.169.0.112 (Modifiable, mais important de le connaître car utilisable dans des prochaines parties)

On active le SSH et RaspberryPi connect (cela sert à accéder au Pi depuis un autre appareil et de le configurer à distance).

Téléchargement du dossier iotcompose :

Une fois que vous avez installé l'image et connecté le Pi à internet, on télécharge un dossier disponible sur github qui contient le dossier de configuration du docker et de différents services. Quelques modifications seront à apporter à quelques fichiers afin de les adapter à la configuration.

Le dossier se trouve ici (téléchargez le fichier et décompressez-le sur le pi) :

<https://github.com/SazaruS1/iotcompose-config.git>

Configuration de la RTC DS1307 :

On ouvre un terminal, et on entre « sudo apt get upgrade » pour faire toutes les mises à jour nécessaires.

Ensuite, on entre ces commandes dans l'ordre :

```
sudo raspi-config
```

Puis on reboot :

```
sudo reboot
```

On vérifie I2C :

```
sudo apt-get install -y i2c-tools
```

```
i2cdetect -y 1
```

On observe UU pour l'adresse 0x68.

Activation du Device Tree Overlay :

```
sudo nano /boot/firmware/config.txt
```

On ajoute la ligne suivante à la fin du fichier :

```
dtoverlay=i2c-rtc,ds1307
```

Puis on fait Ctrl + o pour enregistrer, entrée et Ctrl + x pour quitter.

Puis on reboot.

On vérifie le driver avec :

```
dmesg | grep -i "rtc"
```

et on doit observer :

```
"rtc-ds1307 1-0068: registered as rtc0"
```

Ensuite on synchronise l'heure NTP, on commence par désactiver NTP :

```
sudo timedatectl set-ntp false
```

On définit l'heure système :

```
sudo timedatectl set-time "$(date '+%Y-%m-%d %H:%M:%S')"
```

Puis on réactive NTP :

```
sudo timedatectl set-ntp true
```

On vérifie la synchronisation avec :

```
timedatectl
```

Normalement on voit :

System clock synchronized: yes

RTC in local TZ: no

Dans le terminal, on peut créer une interface minimale qui affiche l'heure au démarrage :

```
nano ~/.bashrc
```

On se déplace à la fin du fichier et on ajoute une portion de code :

```
# Afficher l'heure de la DS1307 au démarrage
```

```
show_time() {  
    echo -e "\033[1;36m=== Heure DS1307 ===\033[0m"  
    local rtc_time=$(timedatectl | grep "RTC time" | awk -F: '{print $2}')  
    local sys_time=$(date '+%a %d %b %Y %H:%M:%S %Z')  
    echo -e "\033[1;32mRTC (UTC)\033[0m : $rtc_time"  
    echo -e "\033[1;33mSys (Local)\033[0m : $sys_time"  
    echo ""  
}  
show_time
```

On fait Ctrl + o pour enregistrer, entrée et Ctrl + x pour quitter et on recharge la config :

```
source ~/.bashrc
```

Et c'est terminé !!

Configuration du Docker Compose :

On va installer docker toujours dans le terminal, puis nous allons nous déplacer dans le dossier que nous avons téléchargé précédemment contenant des fichiers qui nous seront utiles.

Suite des commandes à entrer pour paramétrer Docker :

Installation de Docker :

```
sudo apt-get update
```

```
sudo apt-get upgrade -y
```

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
sudo sh get-docker.sh
```

```
sudo usermod -aG docker simon1 (ou identifiant ça dépend de l'utilisateur).
```

newgrp docker

docker run hello-world

Si il n'est pas installé, installation de Docker Compose :

```
sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

```
sudo chmod +x /usr/local/bin/docker-compose
```

On vérifie avec :

```
docker-compose --version
```

Maintenant on va se déplacer dans le dossier iotcompose :

```
cd ./iotcompose
```

(pour revenir en arrière depuis le terminal, on peut utiliser « cd ../ »)

```
cd ~/iotcompose
```

En entrant la commande suivante, on devrait ouvrir le fichier docker compose :

```
nano docker-compose.yml
```

On peut paramétrer la partie suivante directement depuis le terminal :

Dans la section pihole, on peut ajouter et modifier la liste des adresses statiques.

Dans la partie contenant ces lignes :

```
FTLCONF_dhcp_hosts: |-  
    14:b5:cd:a5:fe:bb,192.168.1.67,PCSimoteSTDocker  
    dc:a6:32:2e:f7:61,192.168.1.2,PI  
    2c:f7:f1:1c:b4:e8,192.168.1.49,gateway-eth  
    20:97:27:2E:C1:0A,192.168.1.3,tap
```

On peut ajouter les adresses MAC, les adresses IP et les noms d'hôtes des machines qui se connectent au réseau. Libre à vous de modifier cette liste. Ici, le tap correspond au Tap200.

Si il contient bien des informations, on fait Ctrl + o pour enregistrer, entrée et Ctrl + x pour quitter.

Ensuite, on peut utiliser la commande :

```
ls
```

Cette commande nous informe des fichiers présents dans le dossier, on doit voir le docker-compose.yml, et d'autres dossiers de configurations qui devraient aussi être présents.

Le dossier configuration contient les configurations de tous les services, on peut y trouver des fichiers de configuration pour pihole, chirpstack, postgresql, mosquito...etc.

Le dossier GYSMO-CONFIG contient les configurations spécifiques à GYSMO.

Depuis le terminal toujours, on lance les mises à jour demandées par Gysmo :

```
sudo apt update
```

```
sudo apt -y update
```

```
sudo apt install -y python3-pip
```

```
sudo apt install -y build-essential libssl-dev libffi-dev python3-dev
```

```
sudo apt install -y python3-venv
```

```
mkdir ENV
```

Pour lancer le docker-compose, il suffit d'entrer depuis le terminal :

```
docker-compose up -d
```

Il faudra attendre un peu le temps que les conteneurs soient créés et lancés, puis on peut vérifier que tout est opérationnel avec :

```
docker ps
```

et pour stopper les conteneurs :

```
docker-compose down
```

Le docker-compose va gérer le bon fonctionnement des différents services, il est au centre de la configuration.

Configuration du Stithub :

On va dans cette étape, établir une courte page html qui va se lancer automatiquement au lancement du Raspberry Pi et qui va nous permettre d'accéder aux différents services lorsque ceux-ci tournent quand le fichier docker compose est up.

L'index Stithub se trouve dans le dossier iotcompose et plus précisément dans le sous dossier www.

Si on se trouve dans ce sous dossier www, on peut accéder au contenu du code de la page :

```
nano stithub.html
```

Pour que le fichier se démarre proprement lors du lancement de chromium (ou d'un navigateur au choix), nous allons créer un fichier au format .desktop dans le dossier autostart d'xdg, pour que le fichier se lance automatiquement au démarrage de la machine. On va donc créer ce fichier avec nano et indiquer le chemin d'accès vers la page html :

```
sudo nano /etc/xdg/autostart/stithub.desktop
```

On copie et on colle ces lignes dans le fichier :

[Desktop Entry]

Type=Application

Exec=chromium file:///home/simon1/iotcompose/www/stithub.html # On remplace par le bon chemin d'accès vers la page

Hidden=false

NoDisplay=false

Name=StigHub Dashboard

On sauvegarde avec ctrl+o, entrer puis ctrl+x et c'est bon.

On redémarre le pi et la page devrait s'ouvrir lors du lancement du navigateur.

Si on modifie les ports depuis le docker compose ou si on change les adresses, on peut modifier directement le fichier stithub.html pour que les liens soient à jour, il suffit de faire un ctrl+f dans le fichier lorsqu'on l'ouvre avec un éditeur de texte graphique comme gedit ou visual studio code et de chercher les ports ou les adresses IP pour les remplacer par les nouvelles.

Paramètres de Pihole et des containers :

Une fois que le docker compose est actif, nous allons pouvoir paramétrer les différents services. Ce paramétrage se fera directement depuis l'interface web des services accessibles depuis Pihole ou en entrant les adresses directement dans le navigateur.

Dans Pihole, on se déplace jusqu'à la partie DHCP, on s'assure qu'il tourne bien, on définit la plage d'adresses IP à distribuer (comme : 192.168.1.50-150), on sauvegarde et on redémarre le service DHCP de Pi Hole.

On définit aussi un temps de lease en cliquant sur le petit bouton vert « basic », 48h suffisent amplement, mais on peut modifier selon sa convenance.

Ensuite, dans le terminal :

On ajoute des règles de firewall pour autoriser le trafic DHCP :

```
sudo apt update
```

```
sudo apt install ufw
```

```
sudo ufw enable
```

```
sudo ufw allow 80/tcp
```

```
sudo ufw allow 67/udp
```

```
sudo ufw allow 68/udp
```

Normalement, le pi possède deux connexions différentes :

Une connexion wifi pour se connecter à internet et effectuer des mises à jour.

Une connexion ethernet établissant une liaison avec le tap200.

Il faut s'assurer que ces deux connexions soient opérationnelles.

Configuration de la priorité du wifi sur ethernet :

Dans les paramètres du Pi, une métrique donne la priorité à la connexion ethernet et non à la connexion wifi. Il est important de modifier cela afin que le Pi dispose d'une connexion wifi active, capable d'effectuer des mises à jour.

On peut observer ces métriques en utilisant ces commandes dans le terminal :

```
ip addr
```

```
ip route show
```

Les métriques doivent être équivalents, si une connexion possède une métrique plus faible qu'une autre, alors celle-ci sera prioritaire que le reste. Il est important de s'assurer que la connexion wifi ne soit pas la connexion avec la métrique la plus haute.

Si le wifi n'est pas prioritaire sur le PI et qu'il a une métrique plus élevée que l'ethernet, la connexion au wifi peut ne pas se faire correctement. Il faut baisser la métrique du wifi pour qu'il soit prioritaire sur l'ethernet.

Pour ce faire, d'abord on fait :

```
ip route show
```

Puis :

```
ls -l /etc/dhcpd.conf
```

```
systemctl is-active NetworkManager
```

Puis :

```
nmcli -t -f NAME,TYPE,DEVICE connection show
```

Puis :

#Wifi Prioritaire

```
sudo nmcli connection modify "Caverne" ipv4.route-metric 100 #Remplacer "Caverne" par le nom de votre réseau wifi
```

```
sudo nmcli connection modify "Caverne" ipv6.route-metric 100
```

#Ethernet pas prioritaire

```
sudo nmcli connection modify "eth0" ipv4.route-metric 600 #Remplacer "eth0" par le nom de votre connexion ethernet
```

```
sudo nmcli connection modify "eth0" ipv6.route-metric 600
```

Puis :

```
sudo nmcli connection down "Caverne"; sudo nmcli connection up "Caverne"
```

```
sudo nmcli connection down "eth0"; sudo nmcli connection up "eth0"
```

Configuration du TAP 200 :

Le Tap200 doit être connecté au Pi via le câble ethernet. Pihole doit le détecter et lui adresser une adresse DHCP statique basée sur celle entrée dans le fichier docker compose.yml.

Depuis stithub on devrait pouvoir accéder à sa page de configuration.